OmegaClaw Runtime Hardening Plan

A proposal for making ProtoMegaBot/OmegaClaw closer to OpenClaw-level operational stability

Prepared by ZeroBot for Benjamin Goertzel and the OmegaClaw hardening team

2026-07-05

Abstract

OmegaClaw has already demonstrated that its symbolic/MeTTa/Prolog/Python architecture can interact with Telegram, local memory, embeddings, and the OpenClaw Gateway. The stability gap relative to ZeroBot/OpenClaw is not primarily a conceptual weakness of OmegaClaw. It is an operational hardening gap: OmegaClaw currently behaves like a long-running research process, while OpenClaw behaves like a supervised runtime with bounded sessions, structured errors, restartable lanes, explicit tool contracts, and health-aware scheduling.

This document proposes a concrete hardening plan. The plan is based on observed OmegaClaw/ProtoMegaBot failures and mitigations during June-July 2026: SWI/Janus crashes during message handling, Telegram polling and resolver-policy failures, context/history blowups, silent backend failures, stale PID behavior, timeout mismatch, environment inheritance problems, and Gateway restart sensitivity. The recommendation is not to turn OmegaClaw into a less interesting system. It is to put OpenClaw-style reliability scaffolding around the symbolic organism so that crashes become localized, diagnosed, and recoverable rather than whole-bot failures.

Contents

1 Executive summary

The current OmegaClaw process is less robust than the OpenClaw/ZeroBot process because it combines too many failure domains inside one live, stateful loop:

- Telegram polling and attachment ingestion.
- MeTTa/PeTTa evaluation.
- SWI-Prolog runtime state.
- Janus Python interop.
- Python libraries for OpenAI/OpenClaw calls, embeddings, Chroma, and file access.
- Local history/memory growth.
- Long-context LLM calls through OpenClaw Gateway.
- Shell-supervised process lifecycle.

By contrast, OpenClaw already has many production-runtime properties: isolated agent turns, structured tool calls, bounded context/session management, model fallback, cron/scheduler semantics, runtime logs, tool policy, and a Gateway process that can keep serving even when an individual tool call or agent turn fails.

The hardening goal should be:

Make OmegaClaw fail small, fail visibly, and recover automatically, while preserving the symbolic cognitive architecture.

The recommended plan has six workstreams:

1. Replace single PID liveness with semantic health checks and restart-on-bad-health.
2. Split the monolithic live loop into supervised, bounded subprocesses.
3. Make every external call return structured success/error data instead of silent empty values.
4. Pin and audit runtime environment, sessions, tokens, model routes, and filesystem policy.
5. Add persistent run records, transcript checksums, cost/token accounting, and crash bundles.
6. Stage live features through non-live, private, and group smoke gates before enabling them in the main bot.

2 Observed OmegaClaw failure modes motivating this plan

This section lists concrete examples observed during the local ProtoMegaBot/OmegaClaw deployment. The goal is not blame assignment; each incident identifies an engineering seam that should become a hardened boundary.

2.1 SWI/Janus crash during message handling

Observed: On 2026-06-27, Ben pinged ProtoMegaBot in the shared Telegram group. The bot received the fresh group message, but SWI/Janus segfaulted during or shortly after constructing the OpenClaw/Python call path before replying. This showed that the main problem was not Telegram delivery alone. The live symbolic/Python call path could crash the process.

Source pointers:

- `memory/2026-06-27.md`, lines 132–134.
- `projects/omegaclaw/PROJECT.md`, current state notes on Telegram/OpenClaw integration.

Hardening implication: Janus/Python/LLM calls should not run in the same crash domain as the Telegram receive loop or the long-lived MeTTa state. A bad Python callback, encoding edge case, native-library crash, or large prompt should kill at most a worker subprocess, not the bot supervisor.

2.2 Prompt/history pollution and large-context crash surface

Observed: After another fresh ping, ProtoMegaBot received the message and died during the OpenClaw prompt call. The constructed prompt was about 51k characters and included polluted stale history with repeated `No response from OpenClaw.` content. Mitigation capped runtime memory fields for Telegram mode and rotated the live history into an archive.

Source pointers:

- `memory/2026-06-27.md`, lines 136–137.

Hardening implication: OmegaClaw needs hard prompt budgets, history compaction, history validity checks, and fail-closed prompt construction. A bad history file should become a diagnostic response and quarantine event, not an unbounded LLM call or process crash.

2.3 Fixed Gateway session context ballooning

Observed: On 2026-06-28, deep-call stalls were traced to a fixed OpenClaw Gateway `user` session for ProtoMegaBot that had grown to roughly `998k/272k` tokens. Fresh unique-user health checks returned normally. The local fix was to use per-call Gateway sessions because OmegaClaw already supplies its own prompt/history.

Source pointers:

- `projects/omegaclaw/PROJECT.md`, current state section.
- `memory/2026-06-28.md`, line 18.

Hardening implication: OmegaClaw should not rely on a shared unbounded backend session for live bot calls. Session identity should be generated per call or per bounded task. If a persistent memory is desired, it should be explicit OmegaClaw memory with quotas and summaries, not accidental Gateway context accumulation.

2.4 Silent backend failures becoming empty values

Observed: `lib_llm_ext.py` previously converted backend failures, timeouts, or empty output into silent `()` values. This made the bot appear alive while losing the real failure cause. Mitigation added user-visible `(send ...)` diagnostics for backend failure, timeout, and empty output, plus preserved traceback detail.

Source pointers:

- `projects/omegaclaw/PROJECT.md`, current state section.
- `memory/2026-06-28.md`, line 18.

Hardening implication: all boundary calls should return structured result objects with status, error class, retryability, elapsed time, model/session id, and bounded diagnostic text. Empty output should not be overloaded as success, no-op, and failure.

2.5 Supervisor killed by nonzero runner exit and stale PID behavior

Observed: The Telegram supervisor's `set -e` behavior let nonzero runner exits kill the supervisor and leave stale process state. A later mitigation changed supervisor behavior so nonzero runner exits no longer killed the supervisor and so the supervisor could relaunch after SWI/Janus crashes.

Source pointers:

- `memory/2026-06-27.md`, lines 134–136.
- `memory/2026-06-28.md`, line 18.

Hardening implication: shell scripts should not be the primary reliability layer unless they are deliberately written as supervisors. Liveness should include stale PID cleanup, child exit classification, backoff, crash-loop protection, and log preservation.

2.6 Timeout mismatches

Observed: Older 180s/240s HTTP/subprocess timeouts were too short for slow GPT-5.5 calls. The fix aligned OpenClaw HTTP and subprocess timeouts to 900s for deep calls. Separately, `OMEGACLAW_TIMEOUT=0` behaved unreliably as infinite in the current wrapper/supervisor; an explicit long timeout such as 86400s was more reliable.

Source pointers:

- `memory/2026-06-27.md`, lines 132–134.
- `memory/2026-06-28.md`, line 18.

Hardening implication: every loop and external call needs documented timeout semantics, aligned across layers. `0 means infinite` should be avoided unless every layer implements it consistently. Prefer finite watchdog budgets plus restartable continuation.

2.7 Telegram adapter and resolver-policy fragility

Observed: Early private/direct Telegram tests failed because Telegram polling DNS failed under the local Landlock policy. The root cause was `/etc/resolv.conf` symlinking into `/run/systemd/resolve`, which the policy did not permit. The local policy was patched to grant read-only access to the resolver path.

Source pointers:

- `projects/omegaclaw/PROJECT.md`, current state and risks.
- `memory/2026-06-27.md`, lines 1–8.

Hardening implication: runtime policy should have explicit preflight checks for DNS, Telegram API reachability, local path access, temp directories, embedding cache, Chroma DB, and Gateway endpoint access. Policy failures should be detected before entering the live loop.

2.8 Gateway restart sensitivity and environment inheritance

Observed: On 2026-07-05, ProtoMegaBot was restarted by watchdog after the OpenClaw Gateway received `SIGTERM` twice. During the restart window, an embedded agent call failed with `EACCES: permission denied, mkdir '/home/ben/.openclaw/agents/main/sessions'`, indicating that some path or environment was resolving `HOME` to `/home/ben` instead of `/home/openclaw`. This was not an OpenAI key reauthorization failure; it was process/runtime/environment fragility around Gateway restart.

Source pointers:

- Telegram discussion with Ben on 2026-07-05.
- Gateway journal excerpts inspected at 2026-07-05 03:13 PDT.

Hardening implication: OmegaClaw should pin `HOME`, `OPENCLAW_HOME` if applicable, session directories, token file paths, and Gateway base URL in a minimal known environment. A Gateway restart should be treated as a retriable backend outage, not as a reason for the Telegram bot process to die.

3 Stability target

The team should define an explicit service-level target for ProtoMegaBot. A proposed target:

1. The Telegram receive loop remains available across routine OpenClaw Gateway restarts.
2. A failed LLM/backend call returns a user-visible diagnostic within a bounded time.
3. A SWI/Janus/native crash is detected and restarted within 60 seconds, with crash evidence preserved.
4. The bot never silently drops a human message after acknowledging it, except where Telegram itself fails.
5. The bot never accumulates unbounded Gateway context, OmegaClaw history, attachment text, or worker transcripts in the live prompt.
6. Each live reply path has a smoke test that can be run without sending a real group message.
7. Restarts are rate-limited and escalated if crash loops occur.

4 Recommended architecture

4.1 Principle: separate the organism from the organs

OmegaClaw's symbolic loop can remain the cognitive center, but operationally risky services should be separated into supervised organs:

Component	Current risk	Proposed boundary
Telegram adapter	Polling/network/DNS/attachment errors can affect live loop	Separate adapter process or thread with durable inbound queue
MeTTa/SWI core	Native crash kills bot	Supervised worker process with crash restart and state checkpoint
Janus/Python calls	Python/native crashes and env leakage	Dedicated bounded subprocess per call or worker pool
OpenClaw/LLM calls	Timeout, context blowup, Gateway restart	Structured RPC client with per-call sessions, retry policy, circuit breaker
Memory/history	Prompt pollution, unbounded files	Versioned bounded memory store with compaction/quarantine
ThreadKeeper/subagents	Long-running child work and queue semantics	Already moving toward bounded queued worker loop; keep separate from live chat path

4.2 Proposed process layout

A robust near-term layout could be:

```
omegaclaw-supervisor
  |
  +-- telegram-ingress-adapter
  |     - polls Telegram
  |     - validates sender/chat/attachments
  |     - writes inbound event records to inbox queue
  |
  +-- omegaclaw-controller
  |     - reads inbox events
  |     - manages conversation/task state
  |     - invokes symbolic worker via RPC/subprocess
  |     - writes outbound send requests
  |
  +-- symbolic-worker(s)
  |     - runs MeTTa/SWI/PeTTa logic
  |     - has per-task time/memory/prompt budgets
  |     - can crash without losing inbox/outbox
  |
  +-- llm-client-worker(s)
  |     - calls OpenClaw Gateway or raw providers
  |     - per-call session id
  |     - structured error returns
  |
  +-- outbox-sender
        - sends Telegram replies
        - records delivery status
```

This does not require a huge rewrite. The first implementation can be a thin file/JSONL queue and subprocess boundary around the current runner.

5 Hardening workstream 1: semantic health checks

5.1 Problem

The current watchdog checks whether a PID exists. That is necessary but far from sufficient. A process can be alive while unable to poll Telegram, unable to call OpenClaw, wedged in Janus, blocked on a huge prompt, or repeatedly returning empty values.

5.2 Recommendation

Add a health checker that reports a compact JSON object, for example:

```
{
  "status": "healthy" | "degraded" | "unhealthy",
  "pid_alive": true,
  "telegram_poll_age_s": 4.2,
  "last_inbound_event_age_s": 120.5,
  "last_successful_reply_age_s": 360.0,
  "openclaw_probe": "ok",
```

```
  "symbolic_probe": "ok",
  "janus_probe": "ok",
  "memory_probe": "ok",
  "queue_depth": 0,
  "last_error_class": null,
  "restart_recommended": false
}
```

The watchdog should restart when semantic health is bad, not only when the PID is dead.

5.3 Specific probes

1. **Telegram probe:** Can call `getMe`; polling offset is advancing or idle without error; send is available in dry-run mode.
2. **OpenClaw probe:** A tiny per-call session prompt returns within a small timeout. Use a non-expensive model route if possible.
3. **Symbolic probe:** A minimal MeTTa/SWI expression evaluates.
4. **Janus probe:** A trivial Python call returns expected value.
5. **Memory probe:** History file exists, is below budget, is parseable, and is not dominated by repeated error strings.
6. **Attachment probe:** Temp directory exists and is writable; configured max bytes/chars are sane.
7. **Queue probe:** Inbound/outbound queues have bounded depth and no stale claimed item.

5.4 Why this is needed

The 2026-06-28 issue where the Gateway session had grown to about `998k/272k` tokens would not necessarily kill the PID, but it made real replies stall. The previous silent `()` behavior also made the bot appear alive while not usefully responding. PID liveness alone cannot detect these cases.

6 Hardening workstream 2: crash containment and subprocess boundaries

6.1 Problem

A SWI/Janus crash can kill the entire Telegram bot. Python calls, native embeddings, Chroma, and provider clients should not share a fatal crash domain with the message receiver.

6.2 Recommendation

Move risky operations behind subprocess or RPC boundaries:

- Keep the Telegram ingress process small and boring.
- Run each human message through a bounded worker invocation.
- Persist the inbound message before invoking the worker.
- Persist the worker result before sending the outbound message.

- If the worker crashes, mark the event as failed, restart the worker, and optionally send a concise diagnostic.

A minimal first version can use command-line subprocesses and JSON files:

```
omegaclaw-run-one-event --event inbox/20260705T...json \
  --result results/20260705T...json \
  --timeout-s 900
```

6.3 Crash bundle

On worker crash, preserve:

- event id and Telegram metadata without secrets;
- bounded prompt digest and prompt size;
- last N log lines;
- process exit code/signal;
- model/session id;
- relevant env hash/allowlisted env dump;
- memory/history file size and checksum;
- core dump pointer if available and safe.

6.4 Why this is needed

The 2026-06-27 SWI/Janus segfault during message handling should have produced a preserved crash record and a restarted worker while Telegram polling remained alive. Instead, the whole live path had to be diagnosed after the fact.

7 Hardening workstream 3: structured error propagation

7.1 Problem

Silent `()` values erased important distinctions:

- backend timeout;
- backend authentication failure;
- context overflow;
- empty model response;
- parser failure;
- local Python exception;
- Telegram send failure.

7.2 Recommendation

Define a standard result envelope for every boundary:

```
{
  "ok": false,
  "stage": "openclaw_call",
  "error_class": "context_overflow",
  "retryable": false,
  "user_visible": true,
  "summary": "Prompt exceeded model context budget.",
  "diagnostic_ref": "runs/20260705T.../error.json",
  "elapsed_ms": 1234,
  "input_chars": 51234,
  "output_chars": 0
}
```

Then map these into MeTTa-level atoms deliberately, for example:

```
(LLMResult Error context_overflow RetryableFalse DiagnosticRef)
```

Do not use one empty atom/list for every failure.

7.3 User-visible failure policy

For live Telegram, a good failure response is short and honest:

> I hit a backend context overflow while forming the response. I saved a diagnostic record and will need a shorter/compacted context before retrying.

This is much better than silence, repeated `No response from OpenClaw.`, or a dead bot.

8 Hardening workstream 4: state and session hygiene

8.1 Prompt budget enforcement

Before every LLM call, compute and record:

- prompt character count;
- approximate token count;
- history contribution;
- memory/RAG contribution;
- attachment contribution;
- tool-result feedback contribution.

If over budget, fail closed into a compaction path:

1. save full raw context to local artifact if safe;
2. produce a bounded summary;
3. retry only if the summary is below budget;
4. record both original and compacted sizes.

8.2 History quarantine

The repeated `No response from OpenClaw.` pollution suggests a simple invariant:

> If recent history is dominated by repeated backend-error text, quarantine it before it enters the next live prompt.

Potential checks:

- maximum history bytes;
- maximum repeated line count;
- maximum error-string density;
- parseability of history file;
- checksum and rotation metadata.

8.3 Per-call Gateway sessions

Keep the already-applied fix as a permanent design rule:

> OmegaClaw supplies its own prompt/history, so Gateway calls should use fresh per-call or per-task sessions unless an explicit bounded Gateway-side memory experiment is being run.

The per-call session id should include a stable bot prefix and event/run id, not a human-readable secret.

9 Hardening workstream 5: environment pinning and preflight

9.1 Problem

The 2026-07-05 `/home/ben/.openclaw` permission error shows that inherited environment and path resolution can break backend calls. Telegram DNS also previously failed because policy did not permit resolver paths.

9.2 Recommendation

Create a single non-secret runtime env manifest and validate it before launch:

```
HOME=/home/openclaw
OPENCLAW_GATEWAY_BASE_URL=http://127.0.0.1:18789/v1
OPENCLAW_MODEL=openclaw/default
OPENCLAW_SESSION_PER_CALL=true
OPENCLAW_HTTP_TIMEOUT=900
OPENCLAW_SUBPROCESS_TIMEOUT=900
CHROMA_DB_PATH=/home/openclaw/research-agent/projects/omegaclaw/repos/PeTTa/chroma_db
HF_HOME=/home/openclaw/research-agent/projects/omegaclaw/local/huggingface
SENTENCE_TRANSFORMERS_HOME=/home/openclaw/research-agent/projects/omegaclaw/local/sentence_transformers
TMPDIR=/home/openclaw/tmp
```

Secrets should remain in a separate `0600` env file, never in the manifest.

9.3 Preflight checks

Before entering the live loop:

1. Confirm `HOME` and any OpenClaw state path resolve under `/home/openclaw`, not `/home/ben`.
2. Confirm the Telegram token file exists, has mode `0600`, and validates with `getMe`; do not print the token.
3. Confirm Gateway health endpoint or tiny chat completion works.
4. Confirm DNS resolution works under the active policy.
5. Confirm required SWI libraries and Python imports load.
6. Confirm embedding model cache is readable.
7. Confirm history and memory files are within configured bounds.
8. Confirm log/artifact directories are writable.

9.4 Minimal child environment

For subprocess workers, pass only an allowlisted environment. This mirrors the ThreadKeeper hardening direction already taken: sanitize PATH, pin HOME, avoid inherited API keys, close stdin, bound output, and use argv-only subprocess calls.

10 Hardening workstream 6: supervision, restart policy, and crash-loop control

10.1 Recommended supervisor behavior

A robust supervisor should:

- preserve logs on restart instead of truncating the only copy;
- rotate logs with timestamps;
- distinguish clean exit, timeout, signal, health-check failure, and crash;
- use exponential backoff after repeated crashes;
- stop restarting and alert a human after a crash-loop threshold;
- write a current status JSON file for external watchdogs;
- support `start`, `stop`, `status`, `health`, `log`, and `diagnose` commands.

10.2 Specific shell-script corrections

The current local supervisor has already improved from the original `set -e` behavior, but further hardening is recommended:

1. Do not truncate `omegaclaw-telegram-private.log` on every restart without first rotating it. The 2026-07-05 restart log began at the restart, erasing direct evidence from the previous crash.
2. Use a lock file to avoid two supervisors starting the same bot.
3. Write `pid`, `start_time`, `env_manifest_sha256`, and `supervisor_version` into a status file.
4. On child exit, preserve exit status and last log lines in an incident record.
5. Add `health` as a separate command returning JSON.

11 Hardening workstream 7: queue-based message handling

11.1 Why queues matter

Queues make crashes recoverable. If Telegram receives a message and the worker crashes before replying, the event should still exist as an inbox record with status `failed` or `retryable`. Without a queue, a message can disappear into a crashed process.

11.2 Minimal inbox/outbox design

Use JSONL or one-file-per-event records:

```
inbox/pending/<event-id>.json
inbox/claimed/<event-id>.json
inbox/done/<event-id>.json
inbox/failed/<event-id>.json
outbox/pending/<reply-id>.json
outbox/sent/<reply-id>.json
outbox/failed/<reply-id>.json
```

Each event should include:

- source channel and chat id;
- sender id;
- message id;
- received timestamp;
- text and attachment metadata;
- untrusted-content marker;
- checksum of raw event bytes;
- processing attempt count;
- last error class.

11.3 Connection to ThreadKeeper work

The ThreadKeeper branch already contains several relevant hardening patterns:

- queue-only dispatch records;
- checksum sidecars;
- atomic claim/done/failed transitions;
- bounded worker loop;
- stop-file handling;
- lock metadata;
- transcript checksums;
- hash-chained compact index records;
- strict schema validation before worker calls;
- retained failed task records.

These patterns should be reused for OmegaClaw's live Telegram event path rather than reinvented ad hoc.

12 Hardening workstream 8: observability and operator UX

12.1 Structured logs

Add JSONL logs for major events:

- supervisor start/stop/restart;
- inbound Telegram event accepted/ignored;
- prompt built;
- LLM call started/ended;
- backend error;
- memory compaction/quarantine;
- worker crash;
- outbound send success/failure;
- health-check result.

Each log should include a `run_id` or `event_id` so one message can be traced across the system.

12.2 Operator commands

Add or document commands equivalent to:

```
omegaclaw status --json
omegaclaw health --json
omegaclaw diagnose --since 1h
omegaclaw replay --event inbox/failed/...
omegaclaw compact-history --dry-run
omegaclaw rotate-logs
omegaclaw preflight --telegram --openclaw --policy
```

12.3 Alerting policy

Alert Ben or the team only for material conditions:

- crash loop threshold reached;
- paid compute/cost exposure;
- credentials or permission boundary problem;
- repeated message delivery failure;
- data-loss risk;
- semantic health unhealthy after restart.

Avoid alerting for acknowledged known blockers or routine restarts that self-heal.

13 Hardening workstream 9: staging gates

Every reliability feature should pass gates before live group operation.

13.1 Gate 0: static and preflight

- Python compile.
- Shell syntax.
- MeTTa/SWI minimal import smoke.
- Env manifest validation.
- Policy path validation.

13.2 Gate 1: non-live local smoke

No Telegram, no real user message. Use a synthetic inbox event and local mock send.

Expected result: one event moves from pending to done, one outbox message is produced, logs and transcript are written.

13.3 Gate 2: private/direct Telegram smoke

Single allowed user, private chat, bounded prompt, cheap model route if possible.

Expected result: fresh ping receives reply; health JSON shows success; no group messages.

13.4 Gate 3: supervised group smoke

Allowed group/channel only, no autonomous extended discussion. Fresh ping or controlled command.

Expected result: reply to source chat/thread, no duplicate sends, no stale pending update replay.

13.5 Gate 4: stress and fault injection

Inject:

- Gateway unavailable;
- Gateway returns context overflow;
- worker timeout;
- Janus worker crash;
- malformed attachment;
- history over budget;
- wrong HOME in environment;
- Telegram DNS failure;
- duplicate supervisor start.

Expected result: no lost event, bounded diagnostic, preserved incident record, and correct restart/backoff behavior.

14 Implementation roadmap

14.1 Phase 1: immediate hardening, 1–3 days

1. Add log rotation before supervisor restart; never destroy the only crash log.
2. Add `health --json` to the supervisor using existing probes.
3. Add env manifest validation, especially `HOME=/home/openclaw` and Gateway state paths.
4. Add prompt/history budget checks before OpenClaw calls.
5. Make backend result envelopes explicit at the Python boundary.
6. Add a small non-live `run-one-event` smoke using a synthetic event.

Deliverable: a supervisor restart preserves logs and health checks catch bad Gateway/session/history states before Ben notices bot silence.

14.2 Phase 2: queue and incident records, 3–7 days

1. Introduce inbox/outbox event records with pending/claimed/done/failed transitions.
2. Persist per-message run records with checksum sidecars.
3. Add crash bundles for worker exits/signals/timeouts.
4. Add retry/backoff policy for retryable backend errors.
5. Add operator `diagnose` and `replay` commands.

Deliverable: a crashed worker no longer loses messages, and a failed Telegram turn has enough evidence for reproducible diagnosis.

14.3 Phase 3: subprocess isolation, 1–2 weeks

1. Move Janus/Python/OpenClaw calls into a bounded subprocess or worker pool.
2. Keep Telegram ingress independent from symbolic worker lifetime.
3. Add memory and CPU limits where feasible.
4. Add fault-injection tests for worker crash, Gateway restart, and bad history.

Deliverable: SWI/Janus or Python crashes no longer kill the Telegram receive loop.

14.4 Phase 4: operational polish, 2–4 weeks

1. Add structured metrics dashboard or periodic compact report.
2. Add circuit breaker for repeated backend/model failures.
3. Integrate ThreadKeeper queued worker loop for long-running tasks behind explicit task contracts.
4. Add high-stakes adjudication gates for actions beyond ordinary replies.
5. Prepare upstreamable patches where useful.

Deliverable: ProtoMegaBot becomes an inspectable, recoverable service rather than a manually nursed research loop.

15 Concrete design examples

15.1 Example: OpenClaw call wrapper

Instead of returning raw text or `()`, the Python/OpenClaw wrapper should return:

```
{
  "ok": true,
  "stage": "openclaw_call",
  "model": "openclaw/default",
  "session_user": "protomegabot-event-20260705T102300Z-a1b2",
  "elapsed_ms": 28450,
```

```
  "input_chars": 18722,
  "output_chars": 2190,
  "text": "..."
}
```

or:

```
{
  "ok": false,
  "stage": "openclaw_call",
  "error_class": "gateway_unavailable",
  "retryable": true,
  "elapsed_ms": 5030,
  "summary": "OpenClaw Gateway connection failed or restarted.",
  "diagnostic_ref": "artifacts/incidents/20260705T.../openclaw-error.json"
}
```

15.2 Example: history budget guard

Before building a prompt:

```
if history_bytes > MAX_HISTORY_BYTES:
    quarantine(history_file, reason="history_over_budget")
    summary = summarize_or_seed_context(history_file)
    if len(summary) > MAX_HISTORY_SUMMARY_BYTES:
        return structured_error("history_compaction_failed")
```

This directly addresses the polluted history and 51k-character prompt incident.

15.3 Example: Gateway restart handling

If OpenClaw returns connection failure or Gateway restart symptoms:

1. Mark the current event attempt as retryable.
2. Sleep/backoff within the worker budget.
3. Retry at most N times with a fresh per-call session.
4. If still failing, send a concise diagnostic and preserve the event for replay.

Do not let a transient Gateway SIGTERM kill the Telegram loop.

15.4 Example: supervisor log preservation

Before restart:

```
if [ -f "$LOG_FILE" ]; then
  ts=$(date +%Y%m%dT%H%M%S%z)
  cp "$LOG_FILE" "$LOG_DIR/omegaclaw-telegram-private.$ts.pre-restart.log"
fi
: > "$LOG_FILE"
```

Better: write each run to a new log file and update a `current.log` symlink.

16 Risks and tradeoffs

16.1 More machinery can obscure the symbolic architecture

Risk: the team could bury OmegaClaw under operational scaffolding.

Mitigation: keep the symbolic core pure where possible, but make boundaries explicit. The scaffolding should be boring and testable.

16.2 Too much restart can hide bugs

Risk: automatic restarts make crashes less visible.

Mitigation: crash-loop thresholds, incident bundles, and alerting. Restart for availability, but preserve evidence for diagnosis.

16.3 Queues can introduce duplicate replies

Risk: retrying after uncertain send status can duplicate Telegram messages.

Mitigation: outbox records should store Telegram message ids after send; retry only when send is known not to have completed, or mark uncertain cases for operator review.

16.4 Per-call sessions can lose useful continuity

Risk: per-call Gateway sessions avoid context blowup but remove accidental continuity.

Mitigation: continuity should come from OmegaClaw's explicit memory/history, with quotas and summaries, not from unbounded Gateway session state.

17 Success criteria

A hardening release should be considered successful when:

1. A forced Gateway restart during an OmegaClaw call produces a retryable structured error or delayed reply, not a dead bot.
2. A synthetic Janus worker crash is recorded and recovered without killing Telegram ingress.
3. A deliberately overlarge history file is quarantined before an LLM call.
4. A wrong `HOME=/home/ben` environment fails preflight before launch.
5. Repeated backend failures do not pollute future prompts with repeated `No response from OpenClaw.`
6. The watchdog can distinguish healthy, degraded, unhealthy, and crash-loop states.
7. A failed event can be replayed from an artifact without asking Ben to resend the original message.

18 Appendix: minimal checklist for the hardening team

Priority	Item	Evidence to require
P0	Preserve logs across restart	Pre-restart log file exists after forced restart
P0	Env preflight pins `HOME=/home/openclaw`	Wrong-HOME test fails closed
P0	Per-call Gateway sessions stay enabled	No fixed session token growth across repeated calls
P0	Prompt/history budget guard	Over-budget history quarantined before provider call
P0	Structured OpenClaw errors	Context overflow returns typed error, not `()`
P1	Semantic `health --json`	Bad Gateway, bad memory, bad Janus each detected
P1	Inbox/outbox event records	Crash leaves replayable failed event
P1	Worker subprocess boundary	Synthetic Janus crash does not kill Telegram poller
P1	Crash-loop backoff	Repeated crash stops restart storm and alerts
P2	Fault-injection suite	Gateway restart, DNS failure, bad attachment, timeout covered
P2	ThreadKeeper integration	Long tasks go through queued audited worker loop
P2	Operator diagnostics	`diagnose --since 1h` links errors to event ids

19 Closing note

OmegaClaw should not be made stable by making it timid. It should be made stable by making its experiments recoverable. The right target is a cognitive system that can still run unusual symbolic/LLM workflows, but where every risky operation has a boundary, every boundary has a typed result, every crash leaves evidence, and every live message is either answered, queued, or explicitly failed with a reason.